



IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 9: RISC-V

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), Fernanda Escobar (fer_jez@uc.cl)

Pregunta 1: Encuesta de Casas de Hogwarts

Usted desea determinar cuál de las cuatro casas de Hogwarts es la mejor basándose en una encuesta a 12 amigos. Cada amigo votó por una opción (1: Gryffindor, 2: Ravenclaw, 3: Hufflepuff, 4: Slytherin).

Implemente un programa en RISC-V que recorra la lista de votos almacenada en memoria, cuente las frecuencias de cada casa y finalmente imprima el número de la casa ganadora.

Resultados: [4, 3, 1, 4, 2, 2, 1, 3, 4, 1, 2, 4]

Paso Intermedio (Python): Antes de programar en ensamblador, revise la lógica en Python para ayudarte a pensar

```
# Lógica intermedia para la encuesta
votos = [4, 3, 1, 4, 2, 2, 1, 3, 4, 1, 2, 4]
# Contadores para casas 1, 2, 3 y 4
conteos = [0, 0, 0, 0]

# Contar frecuencias
for v in votos:
    conteos[v-1] += 1

# Encontrar el ganador (ID de la casa)
max_votos = 0
ganador = 0
for i in range(4):
    if conteos[i] > max_votos:
        max_votos = conteos[i]
        ganador = i + 1

print(ganador)
```

Hint: Utilice la directiva `.word` para almacenar los votos en el segmento `.data`. Dado que cada instrucción y palabra ocupa 4 bytes, recuerde ajustar sus desplazamientos de memoria en consecuencia.

Solución:

```
.data
votos:      .word 4, 3, 1, 4, 2, 2, 1, 3, 4, 1, 2, 4
conteos:    .word 0, 0, 0, 0 # Array de contadores

.text
main:
    la t0, votos      # t0: puntero a votos (temporary)
    la t1, conteos    # t1: puntero a conteos (temporary)
    addi t2, zero, 0   # t2: i = 0
    addi t3, zero, 12  # t3: N = 12 amigos

loop_votos:
    beq t2, t3, init_max # Si i == 12, terminar conteo
    lw t4, 0(t0)         # t4 = votos[i]

    # Calcular dirección: conteos + (votos[i]-1)*4
    addi t4, t4, -1      # Ajustar casa a índice 0-3
    slli t4, t4, 2       # t4 = t4 * 4 bytes
    add t5, t1, t4       # t5 = dirección final del contador

    lw t6, 0(t5)         # t6 = conteo actual
    addi t6, t6, 1       # t6++
    sw t6, 0(t5)         # Guardar conteo

    addi t0, t0, 4       # Siguiete palabra en votos
    addi t2, t2, 1       # i++
    jal zero, loop_votos

init_max:
    addi t2, zero, 0     # Reiniciar i para buscar ganador
    addi s0, zero, -1    # s0: max_votos (saved register)
    addi s1, zero, 0     # s1: casa_ganadora (saved register)
    la t1, conteos      # Reiniciar puntero a conteos

loop_max:
    addi t3, zero, 4
    beq t2, t3, print_res
    lw t4, 0(t1)         # t4 = conteos[i]

    # Si conteos[i] <= max_votos, saltar
    bge s0, t4, next_iter
    addi s0, t4, 0       # s0 = nuevo max_votos
    addi s1, t2, 1       # s1 = casa ganadora (i + 1)

next_iter:
    addi t1, t1, 4       # Siguiete contador
    addi t2, t2, 1       # i++
    jal zero, loop_max

print_res:
    addi a0, s1, 0       # a0: Argumento para imprimir (ID casa)
    addi a7, zero, 1     # a7: Código syscall PrintInt
    ecall

    addi a7, zero, 10     # a7: Código syscall Exit
    ecall
```

Pregunta 2: Explique el código

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_n_m`:

```
.data
n:  .word 37
m:  .word 5
q:  .word 0

.text
main:
    addi sp, sp, -16
    sw   ra, 12(sp)

    la   t0, n
    lw   a0, 0(t0)
    la   t1, m
    lw   a1, 0(t1)

    jal  ra, func_n_m

    la   t2, q
    sw   a0, 0(t2)

    lw   ra, 12(sp)
    addi sp, sp, 16
    addi a7, zero, 10
    ecall

func_n_m:
    blt  a0, a1, ccc

    addi sp, sp, -16
    sw   ra, 12(sp)

    sub  a0, a0, a1
    jal  ra, func_n_m
    addi a0, a0, 1

    lw   ra, 12(sp)
    addi sp, sp, 16
    jalr x0, 0(ra)

ccc:
    addi a0, x0, 0
    jalr x0, 0(ra)
```

Este fragmento representa el cómputo de una función $f(n, m)$. A partir de este:

1. (1.5 pts.) Indique, con argumentos y en términos de n y m , lo que retorna la función $f(n, m)$. Por ejemplo, $f(n, m) = n + m$.

Solución: La función implementa la **división entera por restas sucesivas**, por lo que:

$$f(n, m) = n / m$$

En el código, el registro `a0` contiene el argumento n y el registro `a1` contiene el argumento m . La subrutina `func_n_m` resta repetidamente m a n hasta que el valor de $a0$ es menor

que *a1*, alcanzando el caso base. En cada retorno desde la recursión se incrementa en una unidad el valor acumulado en *a0*, representando la cantidad de veces que *m* cabe en *n*. De esta forma, el resultado final (el cociente entero de la división) queda almacenado en *a0*, y posteriormente se guarda en la etiqueta *q*.

2. (1.5 ptos.) Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V.

Solución: El fragmento anterior **NO respeta** la convención de llamadas de RISC-V. Aunque el programa ejecuta correctamente la división entera y maneja el **stack** sin errores de ejecución, desde el punto de vista de la convención formal incumple reglas esenciales. En particular:

- El registro *a1* es de tipo *caller-saved*. La función *func_n_m* realiza un llamado recursivo sin respaldar previamente este registro, lo que constituye una violación directa a la convención. Al ser una función recursiva, actúa simultáneamente como *caller* y *callee*, por lo que debe preservar el valor de sus argumentos antes del llamado.
- Los registros *a0* y *a1* no se respaldan en el programa principal (*main*) antes de realizar el llamado a la subrutina, a pesar de que ambos pueden ser modificados por esta. Esto también infringe la responsabilidad del *caller*.
- Los registros *t0*, *t1* y *t2* son de tipo *caller-saved* y se utilizan en el programa principal sin respaldo previo. Aunque en este caso no generan un error funcional, formalmente representan una falta a la convención.
- Además, el código emplea los nombres *x0* en lugar de los nombres simbólicos *zero*. Según la convención de llamadas de RISC-V, se exige el uso de nombres simbólicos estandarizados (*zero*, *ra*, *sp*, *a0-a7*, etc.) en lugar de los nombres numéricos (*x0*, *x1*, ...). Este uso constituye una infracción formal de la convención, aunque no afecte la ejecución.

Pregunta 3: Elabore el código

Elabore, utilizando el Assembly RISC-V, un programa que, dado un arreglo `arr` de largo `len` cuyos elementos son enteros de 32 bits (`.word`), determine si este es un “palíndromo” (1) o no (0), es decir, que si se invierte el orden de sus elementos, el arreglo no cambia. El resultado se debe computar en la variable `is_palindrome`.

A continuación, tres ejemplos:

```
arr = [1, 3, 3, 1] → is_palindrome = 1
arr = [1, 2, 3, 2, 1] → is_palindrome = 1
arr = [1, 2, 3, 4, 5, 6] → is_palindrome = 0
```

Puede utilizar el siguiente fragmento de código como base:

```
.data
arr:      .word 1,3,3,1 # Arreglo
len:      .word 4      # Largo del arreglo
is_palindrome: .word -1 # Es palíndromo => TRUE = 1, FALSE = 0
.text
# Su código aquí
```

IMPORTANTE: No es necesario que respete la convención RISC-V en este ejercicio.

Solución: En la siguiente plana, se muestra una solución que utiliza dos índices `i`, `j` tales que inicialmente `i = 0` y `j = len - 1`. Luego, se realiza el siguiente procedimiento iterativo:

- Si `i >= j`, el programa termina y el arreglo se considera palíndromo.
- Si no ocurre lo anterior, se revisa si `arr[i] == arr[j]`. Si no se cumple la condición, el programa termina y el arreglo no se considera palíndromo.
- Si la condición se cumple, se repite la iteración con nuevos índices: `i += 1`, `j -= 1`.

```

.data
    arr:          .word 1,2,5,3,2,1 # Arreglo
    len:          .word 6          # Largo del arreglo
    is_palindrome: .word -1         # Es palíndromo => TRUE = 1, FALSE = 0
.text
    la s0, arr      # Dirección de memoria del arreglo.
    la s1, is_palindrome # Dirección de memoria del resultado.
    addi s2, zero, 4 # Constante para computar dirección de memoria.
    addi s3, zero, 1 # Constante para cargar un 1 en is_palindrome.
    addi t0, zero, 0 # Índice i.
    lw t1, len      # Índice j.
    addi t1, t1, -1
loop:
    bge t0, t1, is_true # Si i >= j, recorrimos todo el arreglo y el arreglo es palíndromo.
    mul a0, t0, s2      # a0 = 4*i
    add a0, a0, s0      # a0 = dirección de memoria arr[i]
    lw a0, 0(a0)        # a0 = arr[i]
    mul a1, t1, s2      # a1 = 4*j
    add a1, a1, s0      # a1 = dirección de memoria arr[j]
    lw a1, 0(a1)        # a1 = arr[j]
    bne a0, a1, is_false # Si a[i] != a[j], el arreglo no es palíndromo
    addi t0, t0, 1      # i += 1
    addi t1, t1, -1     # j -= 1
    jal zero, loop
is_true:
    sw s3, 0(s1)        # is_palindrome = 1
    jal zero, end
is_false:
    sw zero, 0(s1)      # is_palindrome = 0
end:
    addi a7, zero, 10
    ecall

```

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.

